

EXTENDING LARGE LANGUAGE MODELS FOR SPEECH AND AUDIO CAPTIONING

Changli Tang¹, Wenyi Yu¹, Guangzhi Sun¹, Xianzhao Chen²,
Tian Tan², Wei Li², Lu Lu², Zejun Ma², Chao Zhang¹

¹ Department of Electronic Engineering, Tsinghua University, ² ByteDance

tcl20@mails.tsinghua.edu.cn; cz277@tsinghua.edu.cn

ABSTRACT

Multimodal large language models (LLMs) have shown promising visual perception abilities by connecting with image encoders, but their performance on auditory tasks has not yet been widely investigated. Meanwhile, automatic speech recognition (ASR) and automatic audio captioning (AAC) are often achieved with separate systems, resulting in incomplete auditory perception abilities. To fill in these gaps, in this paper, we present the first study that achieves both ASR and AAC by connecting an LLM with auditory encoders. A dual auditory encoder structure is proposed, integrating the Whisper encoder for speech and the BEATs encoder for audio events with a high temporal resolution by using a Q-Former at the window level. Experiments for ASR and AAC are performed correspondingly on the widely used LibriSpeech, GigaSpeech, WavCaps, AudioCaps, and Clotho datasets and yield promising results. In particular, state-of-the-art results are achieved on GigaSpeech, AudioCaps and Clotho. Our model is also able to caption speech and audio events simultaneously from clips with mixed speech and background audio events, which is a step towards more complete machine auditory perception.

Index Terms— Multimodal large language model, automatic speech recognition, audio captioning, dual encoders, Q-Former

1. INTRODUCTION

Recent studies show that a single large language model (LLM), such as GPT-4 [1], PaLM [2], LLaMA [3], and Vicuna [4], can achieve remarkable performance in a number of text processing tasks, which reveals the possibility of having generic text-based cognitive models. This motivates to extend LLMs to perceive audio, image and other modalities and directly obtain information in real-world [5–18].

Regarding vision perception, the Flamingo structure uses extra cross-attention layers to integrate an image encoder with every layer of the [5]. BLIP2 employs the querying Transformer structure (Q-Former) to align the image encoder output space with LLM input space [6]. InstructBLIP improves Q-Former with an instruction-aware variant, enabling Q-Former to achieve task-specific modality alignment [7]. Regarding speech perception, [8] and [9] connect speech encoders with reduced frame rates to LLMs, and achieve integrated multilingual automatic speech recognition (ASR) and speech translation respectively. X-LLM interfaces an LLM with speech, video, and image encoders [10]. SpeechGPT [11] and AudioPaLM [12] use discrete speech tokens to augment the vocabulary of LLMs, which enables both speech perception and speech generation. In addition to speech, audio events are essential types of sounds but are ignored in ASR. Based on automatic audio captioning (AAC), audio understanding and reasoning are introduced and can be handled

by audio-grounded LLMs [13–17]. MU-LLaMA connects a music encoder with LLaMA for music captioning and understanding [18].

Although ASR and AAC are both widely studied, few focus on modelling both tasks together [19]. In this paper, a decoder-only LLM is connected with encoders for joint speech and audio captioning, which is the first multimodal LLM work studying ASR and AAC together to our knowledge. The Vicuna LLM with 13 billion (B) parameters [4], a variant of LLaMA 13B [3] finetuned to follow instructions, is used as the decoder. Regarding the choice of encoders, the Whisper encoder [20] is used to obtain features beneficial for ASR. This speech encoder is combined with the BEATs audio encoder [21] extracting features more useful for AAC, resulting in a novel structure with complementary dual auditory encoders. A window-level Q-Former approach for sequence processing with high temporal resolution is proposed, that fuses the outputs of the two encoders and achieves fine-grained alignment with the LLM input space. Experimental results demonstrate that our proposed structure achieved lower word error rates (WERs) than a strong Whisper ASR baseline on both LibriSpeech and GigaSpeech datasets, and also outperformed the previous state-of-the-art (SOTA) results [22] on GigaSpeech. SOTA results are also achieved for AAC on the widely used AudioCaps and Clotho datasets in the SPIDER metric [23].

In the rest of this paper: Sec. 2 reviews the background knowledge. Sec. 3 presents the proposed method. The experimental setup and results are given in Secs. 4 and 5. We conclude in Sec. 6.

2. BACKGROUND

2.1. Large language models

By pre-training on a rich corpus and finetuned with instructions, LLMs emerge with a variety of abilities for natural language-related tasks even in few-shot and zero-shot cases, and are pivotal elements in the evolving landscape of multimodal machine perception. In the pipeline-based approach, such as HuggingGPT [24] and AudioGPT [25], the LLM serves as the task controller to analyse the text prompts and decomposes the multimodal task, then delegates task-specific audio, visual or other unimodal models to accomplish each subtask and collect the responses.

Although the pipeline-based approach allows handling multimodal tasks, LLMs only interact with the task-specific models via texts and are therefore unlikely to accomplish tasks that require deep interpretation and understanding of the original inputs. In the integrated approach [5–18], LLMs function as the backbone to bind different modalities, enabling machines to perceive and interpret complex data with a depth that approaches human-like understanding. Through training on substantial and diverse datasets, multimodal LLMs can learn to generate outputs that are not only coherent and contextually relevant but also incorporate insights drawn from multiple modalities. This paper studies the integrated approach.

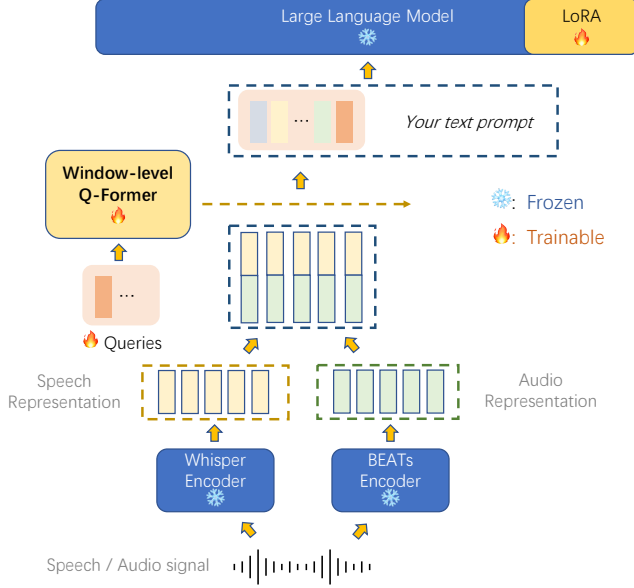


Fig. 1: Our multimodal LLM structure with dual auditory encoders. The Q-Former structure fuses the outputs from Whisper and BEATs encoders for fine-grained alignment with the LLM input space.

2.2. Whisper and BEATs

Whisper [20] is a series of open-source models for multilingual ASR and translation based on the Transformer encoder-decoder structure [26]. By training on a large amount of supervised ASR data, Whisper demonstrates high robustness against variations in speech and background noises. Recently, Gong *et al.* [27] show that the audio representations of Whisper include rich information about background sounds, which makes it suitable for AAC as well.

BEATs [21] uses an acoustic tokeniser to convert input audio to discrete tokens. The tokens are used as labels that are randomly masked and predicted in audio-based self-supervised learning to create a pre-trained model. The tokeniser is then updated by distilling the semantic knowledge from the pre-trained model. These two steps are executed repeatedly and the final model can extract high-level audio semantics information.

3. METHOD

3.1. Q-Former at sequence and window levels

Q-Former is commonly used to connect an encoder with an LLM, which uses a set of trainable queries $\mathbf{H}_0$ to convert an input sequence $\mathbf{Z}$ of length l_{in} to a fixed number of l_{out} text-like tokens [6]. For a Q-Former with b Transformer encoder blocks, each block i contains a multi-head cross-attention layer applied to $\mathbf{Z}$ based on query sequence $\mathbf{Q}_i$, which is generated by a multi-head self-attention layer based on $\mathbf{H}_{i-1}$. By using function $\text{MultiHead}(\cdot)$ defined in [26] to refer to the multi-head attention, $\mathbf{H}_i$ can be calculated as follows:

$$\mathbf{Q}_i = \text{MultiHead}(\mathbf{H}_{i-1}, \mathbf{H}_{i-1}, \mathbf{H}_{i-1}) \quad (1)$$

$$\mathbf{H}_i = \phi(\text{MultiHead}(\mathbf{Q}_i, \mathbf{Z}, \mathbf{Z})). \quad (2)$$

Here, $\phi(\cdot)$ refers to the remaining transforms used in a Transformer block [26] including a small feedforward network, a residual connection with $\mathbf{H}_{i-1}$ and layer normalisation. Denote a Q-Former with b blocks as $\text{Q-Former}_b(\cdot)$ and its output is $\mathbf{H}_b = \text{Q-Former}_b(l_{out}, \mathbf{Z})$.

Regarding speech and audio inputs, the sequence $\mathbf{Z}$ output by the encoder can have a variable length, which is different from hav-

ing image inputs of a fixed size. When connecting auditory encoders with LLMs, the Q-Former can be applied either at the sequence level or window level. At the sequence level, a single Q-Former is used to encode the entire sequence $\mathbf{Z}$ into a sequence $\mathbf{H}$, which consists of l_{token} text-like tokens and serves as the inputs to the LLM. That is,

$$\mathbf{H} = \text{Q-Former}_b(l_{token}, \mathbf{Z}). \quad (3)$$

In this approach, by fixing the number of queries in $\mathbf{H}_0$, Q-Former maps $\mathbf{Z}$ of variable sequence lengths to $\mathbf{H}$ of a fixed length, which reduces the variance of memory usage when processing speech and audio inputs of vastly different lengths.

At the window level, $\mathbf{Z}$ is first segmented into a list of T fixed-size windows $\mathbf{Z}_1, \mathbf{Z}_2, \dots, \mathbf{Z}_T$, then Q-Former can be applied to each window $\mathbf{Z}_t$ simultaneously and independently to generate the relevant token sequence $\mathbf{H}_{b,t}$, and the LLM input token sequence $\mathbf{H}$ is a concatenation of all $\mathbf{H}_{b,t}$. Specifically,

$$\mathbf{H}_{b,t} = \text{Q-Former}_b(l_{out}, \mathbf{Z}_t), \quad (4)$$

$$\mathbf{H} = \mathbf{H}_{b,1}, \mathbf{H}_{b,2}, \dots, \mathbf{H}_{b,T}. \quad (5)$$

This results in $\mathbf{Z}$ of a variable length $T \times l_{out}$, and fewer trainable queries $l_{out} \approx l_{token}/T$ (if $T \times l_{out} \approx l_{token}$) than the l_{token} queries trained in the sequence-level approach. Furthermore, according to Eqn. (4), the text-like tokens in $\mathbf{H}_{b,t}$ depend only on $\mathbf{Z}_t$ that enforces monotonic alignments between $\mathbf{Z}$ and $\mathbf{H}$, whereas the Q-Former in the sequence-level approach allows any alignment. As a result, our proposed window-level approach can have a better temporal resolution in sequence processing than the sequence-level approach. Therefore, we adopt window-level Q-Former in this paper.

3.2. LLM with speech and audio encoders

In order to enable an LLM to achieve end-to-end ASR and AAC, it is essential to interface the LLM with auditory encoders for direct speech and audio perceptions. Since most public available encoders are trained to specialise in either speech or audio event tasks, a Whisper-based speech encoder and a BEATs-based audio encoder are used together in this paper to extract complementary features that are beneficial to both ASR and AAC.

Denote $\mathbf{Z}_{speech}$ and $\mathbf{Z}_{audio}$ as features produced by the speech encoder $\text{Encoder}_{speech}(\cdot)$ and audio encoder $\text{Encoder}_{audio}(\cdot)$ based on an input speech or audio sequence $\mathbf{X}$. $\mathbf{Z}_{speech}$ and $\mathbf{Z}_{audio}$ are synchronised and concatenated frame-by-frame to obtain the fused embedding sequence $\mathbf{Z}$. Such a synchronisation is feasible since both encoders output at a 50 Hz frame rate. More specifically, there are

$$\mathbf{Z}_{speech} = \text{Encoder}_{speech}(\mathbf{X}) \quad (6)$$

$$\mathbf{Z}_{audio} = \text{Encoder}_{audio}(\mathbf{X}), \quad (7)$$

$$\mathbf{Z} = \text{Concat}(\mathbf{Z}_{speech}, \mathbf{Z}_{audio}) \quad (8)$$

where $\text{Concat}(\cdot)$ refers to the frame-by-frame concatenation operation. Next, Eqns. (1)–(5) are used to compute the text-like tokens $\mathbf{H}$ based on $\mathbf{Z}$, using window-level Q-Former. Let $\mathbf{P}$ be the text token sequence used to prompt the LLM, and the final output text sequence $\hat{\mathbf{Y}}$ is generated based on

$$\hat{\mathbf{Y}} = \arg \max_{\mathbf{Y}} P(\mathbf{Y}|\mathbf{P}, \mathbf{H}), \quad (9)$$

where $P(\mathbf{Y}|\mathbf{P}, \mathbf{H})$ is estimated by the backbone LLM (Vicuna 13B [4] in this paper) adapted using low-rank adaptation (LoRA) [28].

Instruction finetuning is performed with both ASR and AAC tasks. The cross-entropy loss is calculated based on $P(\mathbf{Y}|\mathbf{P}, \mathbf{H})$. The parameters of LoRA and Q-Former are updated, while those of the LLM and the speech and audio encoders are kept frozen. The whole model structure is shown in Fig. 1.

4. EXPERIMENTAL SETUP

4.1. Model specifications

We adopt the encoder of Whisper-Large-v2¹ [20] as the speech encoder, the finetuned BEATs² [21] as the audio encoder, and Vicuna 13B³ [4] as the backbone LLM. The output of Whisper and BEATs encoders are 1280-dimensional (-dim) and 768-dim respectively, and therefore the input to the Q-Former is 1280+768=2048-dim. The output of Q-Former is 768-dim and is projected to 5120-dim using a fully connected layer. In all experiments, the Q-Former contains two Transformer blocks sharing the same multi-head self-attention layers and is randomly initialised before training. The rank and scale of LoRA are set to 8 and 4.0. The number of trainable parameters is $\sim 33\text{M}$, $\sim 0.24\%$ of the total parameters of the entire model.

Regarding the hyperparameters of Q-Formers, the number of trainable queries l_{out} and window length l_{in} are required to set a window-level Q-Former. Since window-level Q-Former becomes sequence-level when l_{in} is larger than or equal to the length of encoder output sequence $\mathbf{Z}$. Since Whisper is set to have a known maximum input length of 30 seconds [20], which makes the maximum length of $\mathbf{Z}$ to be 1,500 (with the 50Hz frame rate of Whisper). As a result, window-level Q-Former with $l_{\text{in}}=1500$ is equivalent to sequence-level Q-Former.

4.2. Data and evaluation metrics

Both small-scale and large-scale experiments are conducted. Small-scale experiments in Secs. 5.1 and 5.2 are performed using LibriSpeech [29] train-clean-100 and 28k audio clips selected from AudioCaps [30] for quick proof-of-concept and hyper-parameter selection. Here, the test-clean set of LibriSpeech is used for ASR testing. With the best setting found on small-scale experiments, large-scale experiments in Sec. 5.3 are then performed using 960-hour LibriSpeech, AudioCaps, GigaSpeech [31] 1000-hour M-set, WavCaps [32] (with audio clips longer than 180 seconds removed), and Clotho [33]. Joint speech and audio captioning is studied in Sec. 5.4 with a dataset obtained by mixing clean speech from LibriSpeech with background audio clips randomly selected from AudioCaps.

A number of text prompts are used to instruct the LLM to do corresponding tasks. Example prompts used for ASR and AAC include “Listen to the speech and write down its content” and “Please describe the audio” correspondingly. ASR performance is evaluated using WER. Since the LLM has a very small probability to generate repeatedly, we automatically detect the output sentences and remove the repetitions before testing. For AAC, SPIDER [23] is used for evaluation, which is commonly adopted as a main metric for audio captioning tasks [32]. The performance of joint speech and audio captioning is evaluated using both WER and SPIDER.

5. EXPERIMENTAL RESULTS

5.1. Window-level Q-Former with different encoders

Experiments in this section adopt the small-scale setup. Q-Former at the window level is used to compare three different encoder settings: employing a Whisper encoder for ASR, employing a BEATs encoder for AAC, and assembling both encoders for ASR and AAC.

¹<https://huggingface.co/openai/whisper-large-v2>

²Finetuned BEATs_iter3+ (AS2M) (cpt2)

³<https://huggingface.co/lmsys/vicuna-13b-v1.1>

Encoder	Tasks	%WER↓	%SPIDER↑
Whisper Encoder	ASR	4.51	-
BEATs Encoder	AAC	-	45.0
Dual Encoders	ASR + AAC	4.21	46.8

Table 1: Small-scale results of window-level Q-Former with $l_{\text{in}} = 17$ and $l_{\text{out}} = 1$. %WER and %SPIDER are used for ASR and AAC.

WinLen	l_{in}	l_{out}	#Tokens	%WER↓	%SPIDER↑
30 seconds	1500	90	90	8.11	44.2
10 seconds	500	30	90	6.19	44.8
3 seconds	150	9	90	5.88	46.0
1 seconds	50	3	90	5.00	44.0
0.33 seconds	17	1	88	4.21	46.8
0.16 seconds	8	1	187	3.94	42.1

Table 2: Small-scale dual encoder ASR and AAC results in %WER and %SPIDER with window-level Q-Former of different hyperparameters. The “WinLen” column list the absolute time of the window lengths. Row 1 with $l_{\text{in}}=1500$ is a sequence-level Q-Former.

The l_{in} and l_{out} of the window-level Q-Former are set to 17 and 1 respectively, where the window length is close to 0.33 seconds.

The experimental results are listed in Table 1. With dual encoders, the window-level Q-Former model is able to achieve better performance at both ASR and AAC compared to single-task models, with WER of ASR decreasing relatively 7% and SPIDER of AAC improving nearly 2 percentage points. It is expected that dual encoders can improve both tasks at the same time, since the outputs from the speech and audio encoders can complement each other.

5.2. Q-Formers with different hyperparameters

In this section, a comparison of dual encoder systems with different hyperparameters l_{in} and l_{out} of the window-level Q-Former is conducted based on the small-scale setup. The numbers of text-like tokens (the “#Tokens” column in Table 2) are set to ~ 90 , which is an empirical value found for 30-second audio clips. Note that window-level Q-Former of Row 1 in Table 2 is equivalent to sequence-level Q-Former.

As shown in Table 2, the ASR performance in WERs generally improves when l_{in} reduces and the encoder output sequence $\mathbf{Z}$ is framed with finer granularity. This is possibly due to the fact that window-level Q-Former with smaller window lengths (a.k.a. smaller l_{in}) can produce sequence $\mathbf{H}$ of text-like tokens with higher temporal resolutions, according to Eqn. (4). It is possible that reserving more temporal information may reduce the difficulty in the sequence processing of $\mathbf{H}$ later by LoRA and the backbone LLM. Such a window-to-token generation also reduces the chance of having key information omitted by long-span attention layers in Q-Former.

To verify the assumptions, the effect of having different window granularity coarseness is shown by the case study presented in Fig. 2. A ~ 30 -second speech is first processed to obtain its corresponding tokens in $\mathbf{H}$ by Q-Formers with different l_{in} and l_{out} , then the cosine similarities between these text-like tokens and the embeddings of text tokens in $\mathbf{Y}$ of the ground truth are calculated. The more similar $\mathbf{H}$ and $\mathbf{Y}$ are, the clearer the diagonal line from top left to bottom right will be in Fig. 2, and the easier the LLM and LoRA processing is. This matches our assumptions and the results in Table 2.

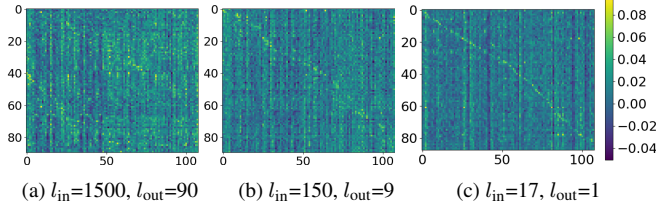


Fig. 2: Cosine similarity analysis of Q-Formers with different window lengths. The vertical and horizontal axes are the index of text-like tokens and embeddings of the ASR ground truth.

Method	%WER↓			%SPIDER↑		
	LS clean	LS other	GS	AC	Clotho	
Whisper-Large-v2	2.47	5.16	9.55	-	-	
SOTA on AAC [32]	-	-	-	48.5	31.0	
Dual Encoders	2.20	5.07	9.20	50.6	31.8	

Table 3: Large-scale ASR results in %WER on LibriSpeech (LS) and GigaSpeech (GS), and AAC results in %SPIDER on AudioCaps (AC) and Clotho. Our Dual Encoders model uses $l_{in}=17$ and $l_{out}=1$.

5.3. Results of large-scale experiments

From Table 2, $l_{in}=17$ and $l_{out}=1$ are used in all experiments with window-level Q-Former thereafter. This setup is later used to train models on larger datasets with our large-scale setup.

Since the captions of the WavCaps dataset included in the large-scale dataset are annotated by ChatGPT⁴, there may be a gap in quality between the annotations provided by ChatGPT and human annotators. Therefore, the procedure of training our models can be divided into two steps. First, the model is pre-train with all speech and audio data, and then the pre-trained model is finetuned on a relatively small amount of data. LibriSpeech 960h, AudioCaps, Clotho, and 200,000 samples randomly selected from GigaSpeech M-set are used for finetuning. Since AudioCaps and Clotho have slightly different audio lengths and annotation styles, we employ different text prompts to generate captions for audio from different datasets when fine-tuning.

By introducing large-scale data, our model is well-equipped for both ASR and AAC tasks. Lower WERs are achieved on both LibriSpeech and GigaSpeech, compared to a strong Whisper baseline, and the previous SOTA result [22] is surpassed on GigaSpeech. Besides, our model also achieves new SOTA results in SPIDER for AudioCaps and Clotho as well. The results are shown in Table 3.

5.4. Joint speech and audio captioning

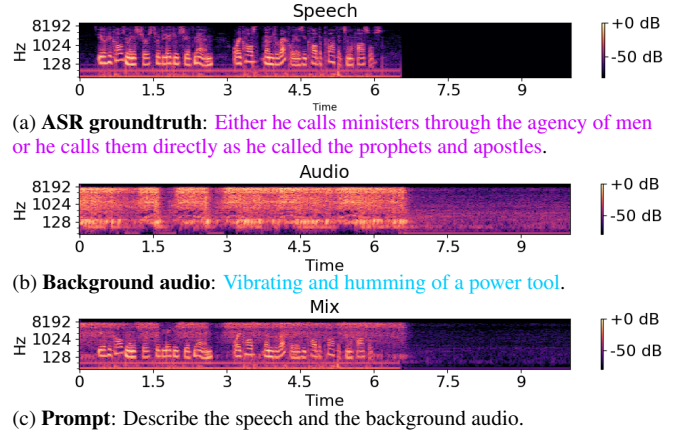
Since speech and audio events often exist together in natural audio scenes, it is an important ability for machines to simultaneously caption and understand not only the speech but also the background audio events mixed with speech. This task was first studied by Narisetty *et al.* [19] that requires achieving ASR and AAC in one model. In this paper, we refer to this task as joint speech and audio captioning and study the task with our proposed multimodal LLMs.

To create a dataset used for the task, audio event data randomly selected from AudioCaps are mixed with the foreground speech from LibriSpeech, where the training, validation and testing sets are mixed from the relevant partitions. We set the signal-to-noise ratio to 3dB since it is a commonly used value in signal processing. The

⁴<https://chat.openai.com>

Method	%WER↓	%SPIDER↑
Whisper-Large-v2	7.32	-
Dual Encoders	6.58	45.0

Table 4: Small-scale ASR and AAC results in % WER and %SPIDER of joint speech and audio captioning. Our Dual Encoders model uses the hyperparameters of $l_{in}=17$ and $l_{out}=1$.



Direct output: Speech: Either he calls ministers through the agency of men or he calls them directly as he called the prophets and apostles. Background audio: A drill is being used.

Prompt: Describe the background audio and the speech in a fluent sentence.
Narrative-style output: While a power tool is being used, the person says 'Either he calls ministers through the agency of men or he calls them directly as he called the prophets and apostles'.

Fig. 3: An example of joint speech and audio captioning.

ASR and AAC references are combined simply by concatenation to form the reference of the new task. To take full advantage of LLMs, narrative-style references with fluent natural language descriptions of both speech and audio background are generated by ChatGPT, which are used to train our model to generate more natural and readable joint captions.

An example of joint speech and audio captioning performed by our model is shown in Fig. 3, which can either generate ASR and AAC outputs separately or together as a narrative-style caption by simply switching to a different prompt. The results listed in Table 4 show that our Dual Encoders model achieves good performance in both WER and SPIDER on the mixed audios.

6. CONCLUSION

In this paper, a Vicuna 13B LLM is extended to connect with the Whisper speech encoder and BEAT's audio encoder for both ASR and AAC based on a proposed dual-encoder structure. A novel window-level Q-Former structure is used to achieve fine-grained alignments between encoder output and LLM input spaces. Our model achieves a 6% relative WER reduction on average on LibriSpeech and GigaSpeech datasets than a strong Whisper ASR baseline, and outperformed the previous SOTA results on GigaSpeech. SOTA results in SPIDER are also achieved for AAC on the widely used AudioCaps and Clotho datasets. Our model is also able to generate narrative-style natural captions based on both speech and background audio simultaneously from mixed audio clips.

7. REFERENCES

- [1] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Rohan A., M. A. Dai, F. Orhan, M. Johnson, D. Lepikhin, et al., “PaLM 2 technical report,” *arXiv:2305.10403*, 2023.
- [3] H. Touvron, T. Lavril, et al., “LLaMA: Open and efficient foundation language models,” *arXiv:2302.13971*, 2023.
- [4] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J.E. Gonzalez, I. Stoica, and E.P. Xing, “Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality,” March 2023.
- [5] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al., “Flamingo: A visual language model for few-shot learning,” in *Proc. NeurIPS*, New Orleans, 2022.
- [6] J. Li, D. Li, et al., “BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *Proc. ICML*, Vienna, 2023.
- [7] W. Dai, J. Li, D. Li, A.M.H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, and S. Hoi, “InstructBLIP: Towards general-purpose vision-language models with instruction tuning,” *arXiv preprint arXiv:2305.06500*, 2023.
- [8] Y. Fathullah, C. Wu, E. Lakomkin, J. Jia, Y. Shangguan, K. Li, J. Guo, W. Xiong, J. Mahadeokar, O. Kalinli, et al., “Prompting large language models with speech recognition abilities,” *arXiv preprint arXiv:2307.11795*, 2023.
- [9] J. Wu, Y. Gaur, Z. Chen, L. Zhou, Y. Zhu, T. Wang, J. Li, S. Liu, B. Ren, L. Liu, and Y. Wu, “On decoder-only architecture for speech-to-text and large language model integration,” *arXiv preprint arXiv:2307.03917*, 2023.
- [10] F. Chen, M. Han, H. Zhao, Q. Zhang, J. Shi, S. Xu, and B. Xu, “X-LLM: Bootstrapping advanced large language models by treating multi-modalities as foreign languages,” *arXiv preprint arXiv:2305.04160*, 2023.
- [11] D. Zhang, S. Li, X. Zhang, J. Zhan, P. Wang, Y. Zhou, and X. Qiu, “SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities,” *arXiv:2305.11000*, 2023.
- [12] P.K. Rubenstein, C. Asawaroengchai, et al., “AudioPaLM: A large language model that can speak and listen,” *arXiv preprint arXiv:2306.12925*, 2023.
- [13] Y. Gong, H. Luo, A.H. Liu, L. Karlinsky, and J. Glass, “Listen, think, and understand,” *arXiv preprint arXiv:2305.10790*, 2023.
- [14] H. Zhang, X. Li, and L. Bing, “Video-LLaMA: An instruction-tuned audio-visual language model for video understanding,” *arXiv preprint arXiv:2306.02858*, 2023.
- [15] Y. Su, T. Lan, H. Li, J. Xu, Y. Wang, and D. Cai, “PandaGPT: One model to instruction-follow them all,” *arXiv preprint arXiv:2305.16355*, 2023.
- [16] Chenyang Lyu, Minghao Wu, Longyue Wang, Xinting Huang, Bingshuai Liu, Zefeng Du, Shuming Shi, and Zhaopeng Tu, “Macaw-llm: Multi-modal language modeling with image, audio, video, and text integration,” *arXiv preprint arXiv:2306.09093*, 2023.
- [17] Yang Zhao, Zhijie Lin, Daquan Zhou, Zilong Huang, Jiashi Feng, and Bingyi Kang, “Bubogpt: Enabling visual grounding in multi-modal llms,” *arXiv preprint arXiv:2307.08581*, 2023.
- [18] S. Liu, A.S. Hussain, C. Sun, and Y. Shan, “Music understanding LLaMA: Advancing text-to-music generation with question answering and captioning,” *arXiv preprint arXiv:2308.11276*, 2023.
- [19] C. Narisetty, E. Tsunoo, X. Chang, Y. Kashiwagi, M. Hentschel, and S. Watanabe, “Joint speech recognition and audio captioning,” in *Proc. ICASSP*, 2022.
- [20] A. Radford, J.W. Kim, T. Xu, G. Brockman, C. Mcleavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. ICML*, Honolulu, 2023.
- [21] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, and F. Wei, “BEATs: Audio pre-training with acoustic tokenizers,” in *Proc. ICML*, Honolulu, 2023.
- [22] Sanchit Gandhi, Patrick Von Platen, and Alexander M Rush, “ESB: A benchmark for multi-domain end-to-end speech recognition,” *arXiv preprint arXiv:2210.13352*, 2022.
- [23] S. Liu, Z. Zhu, N. Ye, S. Guadarrama, and K. Murphy, “Improved image captioning via policy gradient optimization of spider,” in *Proc. ICCV*, Venice, 2017.
- [24] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “HuggingGPT: Solving AI tasks with ChatGPT and its friends in hugging face,” *arXiv preprint arXiv:2303.17580*, 2023.
- [25] R. Huang, M. Li, et al., “AudioGPT: Understanding and generating speech, music, sound, and talking head,” *arXiv preprint arXiv:2304.12995*, 2023.
- [26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. ukasz Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. NIPS*, Long Beach, 2017.
- [27] Y. Gong, S. Khurana, L. Karlinsky, and J. Glass, “Whisper-AT: Noise-robust automatic speech recognizers are also strong general audio event taggers,” in *Proc. Interspeech*, 2023.
- [28] E.J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. ICLR*, 2022.
- [29] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An ASR corpus based on public domain audio books,” in *Proc. ICASSP*, South Brisbane, 2015.
- [30] C.D. Kim, B. Kim, H. Lee, and G. Kim, “AudioCaps: Generating captions for audios in the wild,” in *Proc. NAACL-HLT*, Minneapolis, 2019.
- [31] G. Chen, S. Chai, G. Wang, J. Du, W.-Q. Zhang, C. Weng, D. Su, D. Povey, J. Trmal, J. Zhang, M. Jin, S. Khudanpur, S. Watanabe, S. Zhao, W. Zou, X. Li, X. Yao, Y. Wang, Y. Wang, Z. You, and Z. Yan, “GigaSpeech: An evolving, multi-domain ASR corpus with 10,000 hours of transcribed audio,” in *Proc. Interspeech 2021*, Brno, 2021.
- [32] X. Mei, C. Meng, H. Liu, Q. Kong, T. Ko, C. Zhao, M.D. Plumbley, Y. Zou, and W. Wang, “WavCaps: A ChatGPT-assisted weakly-labelled audio captioning dataset for audio-language multimodal research,” *arXiv preprint arXiv:2303.17395*, 2023.
- [33] K. Drossos, S. Lipping, and T. Virtanen, “Clotho: An audio captioning dataset,” in *Proc. ICASSP*, Barcelona, 2020.